Genetic Algorithms for Various Types of Problems

Slides following
Michalewicz's monograph on Genetic Algorithms (1-3)
Eiben&Smith's monograph on Evolutionary Computing (4)

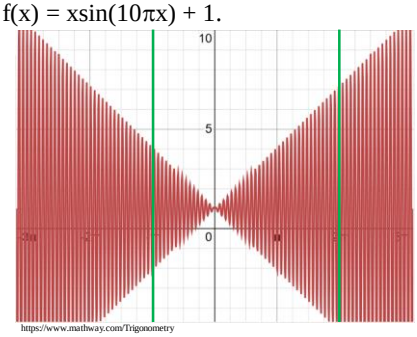
Four Sample GA applications

1. Numerical optimization (continuous)
   - Representing numbers
   - The fitness function is the problem function
2. Combinatorial optimization (discrete)
   - Representing graphs / permutations
   - The objective function becomes fitness
3. Machine learning
   - Representing game strategies
   - Ad-hoc fitness: the "the overall gain"
4. Optimisation with restrictions

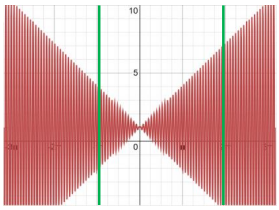
   No given objective function – ad-hoc fitness

A Simple Numerical Example (1)

- Find up to 6 digits $x_0 \in [-1; +2]$ to maximize $f(x) = x\sin(10\pi x) + 1$.



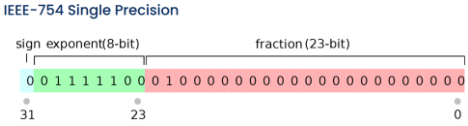
https://www.mathway.com/Trigonometry



- Unconstrained: no maximum – no global best.
- Constrained optimisation problem – one solution.
- Calculus: continuous, derivable
- $f'(x)=10\pi x\cos(10\pi x)+\sin(10\pi x)$

Representation

- Fixed/floating point: **too much** for 6 digits.
- Moreover, applying genetic operators to a chromosome may lead to a representation of a number outside the given interval: **infeasible candidate solutions**.
- And the homeomorphism gives **diverse scales of change** for different loci in the chromosome:



©grainger college, University of Illinois at Urbana Champaign

An ad-hoc representation

- The numerical representations offered by the computer architecture do not allow for two main objectives to be reached:
  - Saving memory
  - **Including constraints in the representation**
- $[-1; +2]$ to be divided into $3\cdot10^6$ equal subintervals.
  - Length-3 interval, 6 decimal digits
- $2097152 = 2^{21} < 3\cdot10^6 < 2^{22} = 4194304$
  - So, a chromosome will have length **m**=22.

- *Decoding*.
  $(b_{21}\, b_{20} \ldots b_0) \rightarrow \sum b_i \cdot 2^i = x' \rightarrow$
  $x = -1 + x' \cdot (3/(2^{22} - 1))$
- There will be 4,194,304 different chromosome instantiations
- While there are only 3,000,000 candidate solutions.
- **Quiz**: what do the remaining 1,194,304 valid chromosomes represent?

A Simple Numerical Example

- (0000000000000000000000) → -1
- (1111111111111111111111) → +2

- v_1=(1000101110110101000111) → x_1= +0.637197
- v_2=(0000001110000000010000) → x_2= -0.958973
- v_3=(1110000000**0**111111000101) → x_3= +1.627888

Evaluation

- $eval(v_1)$ = eval(1000101110110101000111) = $f(x_1)$ = f(+0.637197) = 1.586245;
- $eval(v_2)$ = eval(0000001110000000010000) = $f(x_2)$ = f(-0.958973) = 0.078878;
- $eval(v_3)$ = eval(1110000000111111000101) = $f(x_3)$ = f(+1.627888) = 2.250650

Mutation

- Builds a new candidate solution with **no arithmetic calculation**!

- v'_3 = (1110000000**1**111111000101) →
- x'_3 = +1.630818
- $eval(v_3')$= eval(1110000000**1**111111000101) = $f(x_3')$ = f(+1.630818) = 2.343555
- $eval(v_3')$=$f(x_3')$ > $f(x_3)$ = $eval(v_3)$=2.250650

- **Improvement** with no arithmetic
- Has to be validated by **selection**.

Crossover

- Suppose v_2 , v_3 were selected as parents and 5 was randomly generated as the cutting point.
- v_2 = (00000 | 01110000000010000)
- v_3 = (11100 | 00000111111000101)
- Offspring:
- v'_2=(00000|00000111111000101)→x'_2=-0.99811
- v'_3=(11100|01110000000010000)→x'_3=+1.66602
- $eval(v'_2)$ = $f(x'_2)$ = 0.940865 > $eval(v_2)$ = 0.078878
- $eval(v'_3)$ = $f(x'_3)$ = 2.459245 > $eval(v_3)$ = 2.250650 > $eval(v_2)$ = 0.078878
- This is another way of building (possibly better) new solutions without using arithmetic or logical operators.

Main parameters. Experiments

- *pop_size* = 50; p_c = 0.25; p_m = 0.01.

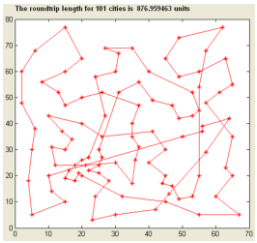
- Solution after 150 generations:
- v_{max} = (1111001101000100000101)
- x_{max} = 1.850773; $eval(v_{max})$ = $f(x_{max})$= 2.850227

- Evolution of $f(x_{best_found_so_far})$ over generations:

1	10	40	100
1.441942	2.250363	2.345087	2.849246

A Combinatorial Problem

- **Travelling Salesperson Problem**. Given the complete graph of **n** cities and the cost of travelling between any two of these cities, find the least expensive itinerary which passes exactly once through each city and returns to the starting one.



Aravind Seshadri (2020). Traveling Salesman Problem (TSP) using Simulated Annealing (https://www.mathworks.com/matlabcentral/fileexchange/65712-travelling-salesman-problem-tsp-using-simulated-annealing), MATLAB Central File Exchange. Retrieved October 25, 2020

Solving approaches

- Exact algorithms:
  - Dynamic programming - Held-Karp algorithm $O(n^2 * 2^n)$
  - Branch and bound – feasible only for < 100 cities
  - Linear programming – feasible for < 200 cities
- Approximate algorithms:
  - Nearest neighbour, Christophides algorithm, Markov-chain algorithms
  - Genetic Algorithms, Simulated annealing, Tabu search, Ant colony optimisation.
- Exact methods solved < 100,000 cities
- Approximate methods solved >1,000,000 cities.
- Obvious approach: optimal **permutation**
  - Exact: exhaustive search **O(n!)**. Infeasible, even for n=20
  - Approximate: Genetic Algorithm **O(n^2)**

A GA for the optimal permutation

- **Representation**: integer vectors for **permutations**:
  - (5 2 4 1 3) for n=5 cities. The tour ends in city 5.
- **Evaluation**: sum of costs of successive edges in the tour.
- **Mutation** on only one gene is not possible.
  - Simply replacing 2 by 3 is not valid:
  - (5 **2** 4 1 3) → (5 **3** 4 1 3). Not a permutation
- Reason: *allele* **restrictions**
- Minimal valid change: switch two randomly chosen genes.
  - (5 **2** 4 1 **3**) → (5 **3** 4 1 **2**)
    - actually, repairing the non-admissible solution

Crossover

- Standard crossover is not possible
  - (5 2 ⋮ 4 1 3)
  - (3 1 ⋮ 5 4 2)

  → ~~(5 2 5 4 2)~~ ~~(3 1 4 1 3)~~
  - Many original crossover operators have been proposed.
- Crossover (OX): preserves **order**.
  - Reminds **grafting** of fruit-trees.

P: (1 2 3 **4 5 6 7** 8 9 10 11 12) (7 3 1 **11 4 12 5** 2 10 9 6 8)

O: (3 1 11 **4 5 6 7** 12 2 10 9 8) (1 2 3 **11 4 12 5** 6 7 8 9 10)

- Copy a subsequence from one parent, for the rest preserve the relative order from the other parent.

Constraints in representation

- Permutations can be represented in a way which allows for standard mutation and standard crossover to be applied.
- (5 2 4 1 3) (1 2 3 4 5)
- (5 2 3 1 1)
- Mutation:
- (5 2 4 1 3)→(5 **2** 3 1 1)→(5 **3** 3 1 1)→(5 3 4 1 2)
- Crossover:
- (5 2 4 1 3)→(5 2 ⋮ 3 1 1)
- (3 1 5 4 2)→(3 1 ⋮ 3 2 1)

  → (5 2 3 2 1)→(5 2 4 3 1)
  (3 1 3 1 1)→(3 1 5 2 4)

Quiz

Please answer "Yes" or "No" (and give reasons for your answer) to the following question:

For the new representation of permutations, does the standard mutation always lead to the actual switch of two original positions in the original permutation?

A simple game: the Prisoner's Dilemma

- A paradox in Decision Theory.
- Two players, **separated and unable to communicate**, trying each to maximise the personal profit, cannot reach the optimal outcome.
  - Trying to obtain a personal advantage at the expense of the other.
- The resulting situation is worse than what they could have obtained by cooperating in the decision-making process.

» Jim Chappelow (University of Southern New Hampshire)

Instantiations of the PD Game

- Many aspects of real economy
  - the use of natural resources,
  - cartel pricing,
- behaviourial sciences
  - evolution of cooperation,
- diplomatic negotiations
  - arms' race,
- …

The Prisoner's Dilemma

Symmetric gain table:

Player1	Player2	Points1	Points2
Defect	Defect	1	1
Defect	Cooperate	5	0
Cooperate	Defect	0	5
Cooperate	Cooperate	3	3

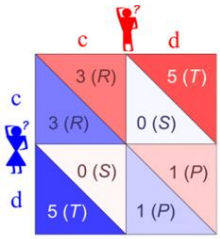
If T>R>P>S, then Nash equilibrium at (D,D)

Player2 moves D→maxPoints1 for Player1 moving D

Player2 moves C→max Points1 for Player1 moving D

Which is worse than (C,C).

Nothing for GA so far.



Players X (blue) and Y (red) each choose to cooperate (c) or defect (d) with respective payoffs R, T, S, or P

©William H. Press (University of Texas) and Freeman J. Dyson (Princeton University) – Proc. of the National Academy of Science

Iterated Prisoner's Dilemma

- Things change spectacularly when the two players repeat the game many times: Iterated Prisoner's Dilemma.
- With the rules of this game, cooperation cannot be learned in one step, but starts being attractive in repeated steps.
- **Problem: find an optimal strategy for a player in the Iterated Prisoner's Dilemma game.**
- No fitness function in the problem statement.
- The fitness will be the total gain of the player.

A Brief Analysis

- Suppose a player would have a constant strategy – say, always D. (S)he will not gain **5*#iterations** points, since the opponent, even if started with C, would change the option.
- Nor constantly C, since the opponent could then play D indefinitely.
- At each iteration, the optimal move of a player depends on the **previous** behaviour of the other player.
- Therefore, an optimal strategy is dynamic - this can be seen as a **dynamic optimum** problem:

the optimal decision cannot be expressed through constants, but rather as a function of the way the competitor plays.

- So, an evolutionary method should be appropriate.

What is a strategy?

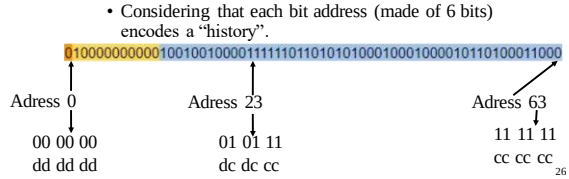
- A strategy is a complete answer to the question: "what is my next move?"
- Complete answer means: a specific answer for any configuration of the game.
- What is a configuration?
- Non-iterated: only one configuration.
- Iterated: a number of configurations depending on the number of **previous moves** taken into consideration.
- Previous moves include the player's own moves, since the opponent's decisions were influenced by them.

IPD: A Genetic Algorithm Approach

- Axelrod (1987).
  - Studied the way cooperation evolves (*Science*).
  - Organised two computer competitions to find out the best strategy.
- Deterministic strategies, based on **previous 3 moves**.
- All **4 possibilities** (configurations) for each move have to be taken into consideration.
- $\mathbf{4^3 = 64}$ different histories before the current move.
- Strategy: what move to make for each possible history. $\mathbf{2^{64}}$ different strategies – huge search space.

IPD – A Genetic Algorithm

- **Representation** of a candidate strategy:
  - There are 64 moves in a strategy – one for each of the 64 configurations
  - A move can be represented by one bit (0/1 instead of C/D)
  - The minimal representation of such a strategy uses 64 bits.
    - Considering that each bit address (made of 6 bits) encodes a "history".



Further bits

- In further experiments, strategies are to be tested against specific, **dynamic** histories.
- Such specific histories can be represented in the chromosomes
- 6 extra bits for the 3-move history of an actual iterated game.
- 70 bits represent a chromosome
- The actual search space has cardinality 2^{70}
  - Coding not only the strategy itself, but also a way of sampling this search space.

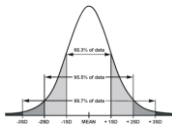
IPD – a Genetic Algorithm

- Random initialisation of the population;
- One generation:
  - Evaluate each player (chromosome) against each of the other players, on their respective games
    - Fitness: the total score of the player in these games
  - Selection for applying operators

    (average score→breed once;

    one SD above average→breed twice;

    one SD below average→no breeding);
    - SD stands for *standard deviation*



  - Standard mutation and crossover on bitstrings.
  - *Roulette-wheel* selection with the probability field described above.

Human Tournaments

- The ***Tit_for_tat*** strategy won two tournaments for human players:
  - cooperate;
  - while (not halt) do what the opponent did in the previous step.
- This strategy's characteristics:
  - be nice - not the first one to defect;
  - be provocable – react adequately;
  - don't be greedy – maximize globally, not locally;
  - don't be too clever – don't try to get occasional extra-benefits, have predictable reactions.

First Round of GA Experiments

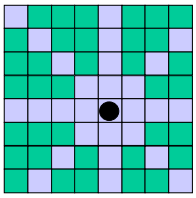
- GA experiments – first round:
  - fixed environment
  - **64-bit chromosomes.**
  - ***pop_size* = 20; *max_number_of_gen* = 50**; *number_of_runs* = 40;
  - evaluation of each chromosome: average of **fixed** iterated games against a **fixed** set of **8 best human strategies**;
- GA results: ***Tit_for_tat*** (or better) in the static *fitness landscape* with the 8 human strategies
  - Explanation of "**better**": ***Tit_for_tat*** is general; some GA solutions were over-adapted to the specific environment.
- **The global optimum was found visiting at most $\mathbf{20*50=1000 \approx 2^{10}}$ strategies out of $\mathbf{2^{64}}$**

which is $\mathbf{1/2^{54} \approx 10^{-17}}$ of the search space!

Second Round of GA Experiments

- *Changing environment.*
  - 70-bit chromosomes
  - each individual evaluated against all 20 (including itself!);
  - **environment changes as individuals evolve**;
  - fitness: average score of the individual in the 20 games.
- Early generations: uncooperative strategies;
- after 10-20 generations: more and more cooperation in the ***Tit_for_tat*** manner: reciprocation!
- Even faster convergence!
- Reason: better sampling of the strategies space.
- Further reading: Martin Nowak.

4th Example: the 8 queens problem



Problem: place 8 queens on an 8x8 chessboard in such a way that they cannot check each other

- Problem stated in 1848 by Max Bezzel.
- Gauss also studied the problem.

Optimisation with restrictions

- Total number of candidate solutions? (feasible or unfeasible)
- 4,426,165,368 = C_{64}^{8}
- **Lemma: Each column, one queen!**
  - Proof – instant.
  - Hence, 16,777,216=8^8 column_wise feasible solutions.
- **Lemma: Each row, one queen!**
  - Which reduces the number of column_and_row_wise feasible solutions to 40,320=8!
    - Diagonal checks still to be eliminated.
- So, a possible representation for a GA would be through **permutations** indicating which row in each column.
    - Only diagonal checks have to be verified.
- Total number of correct solutions is 92.

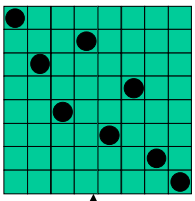
A Tentative Representation

- A candidate solution can be represented as consisting of 8 genes.
- Each gene is a pair (i,j), 1<=i,j<=8
- with the restriction that all 8 numbers on the first position give a valid permutation and the same property for the numbers on the second positions.
- Example: (3,6)(7,3)(2,1)(4,4)(8,5)(1,2)(6,8)(5,7)

A Better Representation

Illustrated for a non-optimal solution

Phenotype:
a board configuration



Obvious mapping

Genotype:
a permutation of
the numbers 1 - 8

1	3	5	2	6	4	7	8

8 Queens Problem: Fitness evaluation

- *Penalty* of one queen:
  the number of queens she can check.
- *Penalty* of a candidate solution:
  the sum of the penalties of all queens.
- *Penalty* has to be minimized (down to 0).
- Fitness of a candidate solution:
  inverse of *penalty+1*, which has to be maximized

The 8 queens problem: Mutation

Small variation in one permutation, e.g.:
- swapping values of two randomly chosen positions

1	3	5	2	6	4	7	8

→

1	3	7	2	6	4	5	8

The 8 queens problem: Recombination

Combining two permutations into two new permutations:
- choose random crossover point
- copy first parts into children
- create second part by inserting values from the other parent:
  - in the order they appear there
  - beginning after crossover point
  - skipping values already copied in the offspring

1	3	5	2	6	4	7	8
8	7	6	5	4	3	2	1

→

1	3	5	4	2	8	7	6
8	7	6	2	4	1	3	5

Inventing New Selection Procedures

- A peculiar **parent selection** procedure:
  - Pick 5 parents and take best two to undergo crossover
- **Survivor selection** (replacement)
  - When inserting a new offspring into the population, choose an existing member to replace by:
    - sorting the whole population by decreasing fitness
    - enumerating this list from high to low
    - replacing the chromosome with the highest fitness lower than the fitness of the given offspring.

8 Queens Problem: summary

Representation	Permutations
Recombination	"Cut-and-crossfill" crossover (OX)
Recombination probability	100% w.r.t. the number of offspring
Mutation	Swap
Mutation probability	80% discuss at the seminar
Parent selection	Best 2 out of random 5
Survival selection	Replace worst
Population size	100
Number of Offspring	2
Initialisation	Random
Termination condition	Solution or 10,000 fitness evaluation

Obviously, this is ***only one possible*** set of choices of operators and parameters

Convergence issues

Typical behaviour of a GA

Phases in optimising on a 1-dimensional fitness landscape

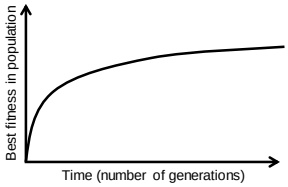
Early phase:
quasi-random population distribution

Mid-phase:
population arranged around/on hills

Late phase:
population concentrated on high hills

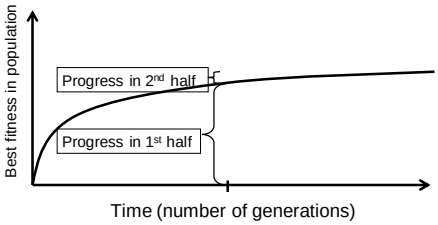
Important: appropriate halting condition!

Typical run: progression of fitness



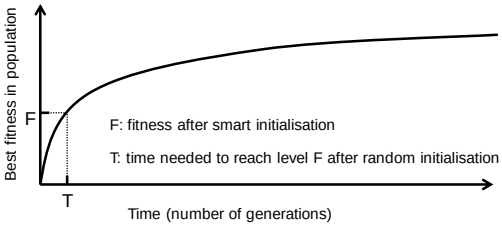
Typical run of a GA shows the so-called "anytime behavior"

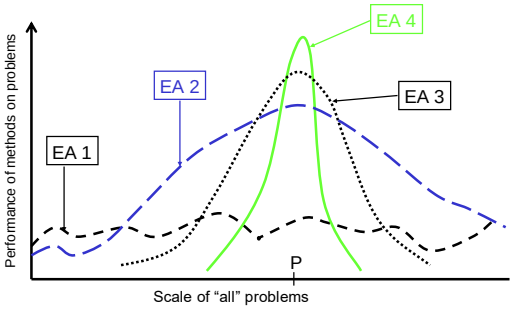
Are long runs beneficial?



- Answer:
  - it depends how much you want the last bit of progress
  - it may be better to do more shorter runs ("iterated", following the hill-climbing approach)

Is it worth expending effort on smart initialisation?



Answer: possibly, if good initialisation methods exist which take less than time T.

Michalewicz' 1996 view